

SOFTWARE REQUIREMENTS SPECIFICATION

NeoBank Platform — Sprint 4 — Insights & Admin Controls

Project	NeoBank Platform
Document	Software Requirements Specification (SRS)
Sprint	Sprint 4 — Insights & Admin Controls
Duration	10 Working Days (Days 31–40 of 60-Day Lifecycle)
Program	PMIS (Pradhan Mantri Internship Scheme) — Infosys Limited
Location	Bhubaneswar Development Centre — Lab 4 & Lab 5
Version	1.0 — Draft, Internal Review
Date	May 2026
Status	Active — Sprint In Progress
Previous Sprint	Sprint 3 — Loan Management (Days 21–30)
Next Milestone	Milestone57 — Stabilization & Deployment

© Infosys Limited · Confidential · Not for External Distribution

1. INTRODUCTION

The NeoBank Platform is an enterprise-grade digital banking solution developed under the PMIS (Pradhan Mantri Internship Scheme) Internship Program at the Infosys Bhubaneswar Development Centre, Lab 4 and Lab 5. This document is the authoritative Software Requirements Specification (SRS) for Sprint 4, covering Days 31 through 40 of the 60-day development lifecycle.

With user onboarding, core banking ledgers, personal finance tooling, and credit facilities fully operational across Sprints 1 through 3, this sprint shifts focus toward data aggregation and platform oversight. Sprint 4 delivers actionable financial analytics for end-users and comprehensive system monitoring dashboards for platform administrators — together forming the intelligence and control layer of the NeoBank Platform.

1.1 Purpose

This SRS defines the complete set of functional requirements, system behaviour, business rules, integration strategy, quality assurance plan, and sprint-level execution roadmap for NeoBank Sprint 4. It serves as the binding technical reference for all interns, mentors, and stakeholders in Lab 4 and Lab 5.

1.2 Scope of Sprint 4

Sprint 4 is scoped exclusively to the Insights and Admin Controls module. Deliverables are limited to:

- FR-6: Insights and Admin Controls — user-specific financial summaries and trend analytics, admin platform dashboards, system monitoring, and pending approval management
- Backend aggregation engine: complex SQL aggregation queries and optimised Spring Data JPA logic for calculating totals and trends across existing transaction, loan, and user data
- Frontend analytics UI: Angular charting components (Chart.js / ng2-charts) rendering income vs. expense trends and savings summaries
- Admin dashboard: secure admin-only views for platform-wide metrics, pending approvals, and user status management
- Unit and end-to-end testing of all sprint deliverables with strict security validation of admin endpoints
- Swagger/OpenAPI documentation updated for all new /api/insights/... and /api/admin/... endpoints

1.3 Project Context

Internship Program	PMIS (Pradhan Mantri Internship Scheme)
Organisation	Infosys Limited
Development Centre	Bhubaneswar DC
Labs	Lab 4 & Lab 5
Project	NeoBank Platform
Sprint Duration	10 Working Days (Days 31–40)
Total Cycle	60 Working Days
Previous Sprint	Sprint 3 — Loan Management (Days 21–30)
Next Sprint	Milestone 5 — Stabilization & Deployment

1.4 Definitions & Abbreviations

TERM	DEFINITION
SRS	Software Requirements Specification — this document
FR	Functional Requirement — a categorised set of deliverable features
JWT	JSON Web Token — signed stateless token used for API authentication
RBAC	Role-Based Access Control — access restrictions based on assigned roles
API	Application Programming Interface — RESTful HTTP endpoints
DTO	Data Transfer Object — decouples API payload structure from JPA entity
KPI	Key Performance Indicator — platform-wide metric surfaced on the Admin Dashboard
PMIS	Pradhan Mantri Internship Scheme — government internship program hosted by Infosys
DC	Development Centre — Infosys Bhubaneswar facility
ORM	Object-Relational Mapping — Spring Data JPA maps Java entities to MySQL
DDL	Data Definition Language — SQL scripts to create and extend database schema

2. FUNCTIONAL REQUIREMENTS OVERVIEW

Sprint 4 delivers one core Functional Requirement module that adds the intelligence and oversight layer to NeoBank. FR-6 aggregates data across all prior modules — accounts, transactions, budgets, bills, rewards, and loans — and must be fully functional and tested before sprint sign-off.

2.1 FR-6: Insights and Admin Controls

This module handles the generation of user-specific financial summaries and trends, establishes secure admin dashboards, and manages overarching system monitoring and approvals. It introduces no new database tables; all analytics are derived from existing data through optimised aggregation queries.

2.1.1 Financial Insights Engine

- ▶ Authenticated users retrieve their personal financial summary via GET /api/insights/{userId}
- ▶ Response payload includes: totalIncome (sum of all CREDIT transactions), totalExpense (sum of all DEBIT transactions), savings (totalIncome – totalExpense), and trendSummary (monthly breakdown of income and expense)
- ▶ Aggregation logic must strictly exclude transactions belonging to inactive accounts (is_active = false)
- ▶ trendSummary provides a month-by-month array (minimum last 6 months) for chart rendering — each entry includes: month label, totalIncome, totalExpense
- ▶ Cross-user access is prohibited — JWT userId must match the path userId; HTTP 403 returned on mismatch
- ▶ Insights are read-only — no POST, PUT, or DELETE operations exist on the insights resource

2.1.2 User Analytics Dashboard (Frontend)

- ▶ Angular component renders the financial summary as visual cards: Total Income, Total Expense, and Savings
- ▶ Chart.js / ng2-charts bar or line chart renders the trendSummary month-by-month income vs. expense overlay
- ▶ Chart dynamically updates on data refresh — no full page reload required
- ▶ Charts must render correctly across standard desktop screen widths and scale responsively
- ▶ Loading skeleton shown while GET /api/insights/{userId} is in-flight

2.1.3 Admin Dashboard Engine

- ▶ Admin users retrieve platform-wide metrics via GET /api/admin/dashboard
- ▶ Response payload includes: totalUsers (all registered users), totalActiveUsers, totalLoans (all loan applications), pendingApprovals (count of PENDING loan applications and any other pending items), totalTransactions, and platformSavingsRate
- ▶ Endpoint is protected with @PreAuthorize("hasRole('ADMIN')") — CUSTOMER role access returns HTTP 403 Forbidden
- ▶ Inactive accounts are excluded from all financial metric calculations on the admin dashboard
- ▶ Admin dashboard metrics are computed on-demand — no caching layer required for this sprint

2.1.4 System Monitoring & Pending Approvals

- ▶ Admin users retrieve detailed pending approval lists via GET /api/admin/pending-approvals
- ▶ Response includes actionable items across modules: pending loan applications (with applicant name, product, amount), pending user verifications if applicable
- ▶ Admin users can update user account status (activate / deactivate) via PATCH /api/admin/users/{userId}/status
- ▶ Only ADMIN role may execute user status updates — CUSTOMER access returns HTTP 403 Forbidden

- ▶ System health monitoring endpoint GET /api/admin/system-health returns: database connectivity status, active session count, and uptime metrics

2.1.5 Advanced Admin Controls

- ▶ Admin users can view all registered users with account details via GET /api/admin/users
- ▶ Admin users can view detailed activity for any user via GET /api/admin/users/{userId}/activity
- ▶ All admin control endpoints are protected with @PreAuthorize("hasRole('ADMIN')") — unauthorised access returns HTTP 403
- ▶ Admin actions are audit-logged — each admin operation records the acting admin's userId, action type, target resource, and timestamp

3. BUSINESS & VALIDATION RULES

The following rules are hard constraints enforced at the backend API layer, independent of any frontend validation. Any API response that violates these rules is classified as a defect.

FR-6 — DATA PRIVACY RULES

REF	CONSTRAINT
BR-01	Financial insights must be visible only to the owning user. The JWT userId must match the path {userId} parameter. Mismatches return HTTP 403 Forbidden before any aggregation query executes.
BR-02	Aggregated data metrics must strictly exclude all transactions and balances from accounts where is_active = false. Including inactive account data is classified as a defect.
BR-03	The trendSummary must cover a minimum of the last 6 calendar months, even if some months have zero transactions. Missing months must be represented with zero values, not omitted.
BR-04	All secured insight endpoints require a valid, non-expired JWT Bearer token in the Authorization header. Missing or invalid tokens return HTTP 401 Unauthorized.
BR-05	Insights are read-only resources. No POST, PUT, PATCH, or DELETE endpoints shall exist for the /api/insights/... resource. Any such request returns HTTP 405 Method Not Allowed.
BR-06	Savings calculation must always use the formula: savings = totalIncome – totalExpense. A negative savings value (totalExpense > totalIncome) is valid and must be returned as-is, never clamped to zero.

FR-6 — ADMINISTRATIVE RULES

REF	CONSTRAINT
BR-01	All /api/admin/... endpoints must be completely restricted to users possessing the ADMIN role. CUSTOMER or unauthenticated access returns HTTP 403 Forbidden or HTTP 401 Unauthorized respectively.
BR-02	Admin dashboard metrics (totalUsers, totalLoans, pendingApprovals) must reflect the live database state at the time of the API call. Stale or cached values that misrepresent the current state are classified as defects.
BR-03	User status updates (activate / deactivate) via PATCH /api/admin/users/{userId}/status must be validated: an admin cannot deactivate their own account. Such requests return HTTP 400 Bad Request.
BR-04	All admin write operations must be recorded in an audit log with: actingAdminId, action, targetResourceType, targetResourceId, and timestamp. Missing audit records are classified as defects.
BR-05	The pendingApprovals count on the admin dashboard must aggregate across all modules that have a PENDING state — including loan applications. The count must be accurate to the second.
BR-06	Admin user listing (GET /api/admin/users) must never return password hashes or any sensitive credential fields in the response payload. Exposing such fields is a critical security defect.

4. INFRASTRUCTURE & ARCHITECTURE

Sprint 4 extends the same modern, loosely-coupled, three-tier full-stack architecture established in Sprints 1 through 3. No new database tables are introduced; analytics are derived entirely from existing schema data through optimised aggregation queries. Role-based access controls are enforced at the Spring Security layer.

4.1 Technology Stack

LAYER	TECHNOLOGY	DETAILS
Frontend	Angular	TypeScript SPA; Angular CLI; Chart.js / ng2-charts for trend visualisation; Angular Material cards for KPI summary display; HTTP Interceptors; Route Guards for admin routes
Backend	Spring Boot	Java REST API; Spring Security with <code>@PreAuthorize</code> for all Admin endpoints; Spring Data JPA with <code>@Query</code> for complex aggregations; Maven; <code>@Transactional(readOnly = true)</code> on all analytics reads
Database	MySQL 8.0+	Existing schema from Sprints 1–3; no new DDL scripts required; complex GROUP BY and SUM aggregation queries on transactions, accounts, users, loan_applications tables
Security	JWT + BCrypt	Stateless token auth inherited from Sprint 1; <code>@PreAuthorize("hasRole('ADMIN')")</code> on all <code>/api/admin/...</code> endpoints; JWT <code>userId</code> cross-check on <code>/api/insights/{userId}</code>
Version Control	Git	Feature-branch workflow; Sprint 4 branches from develop; all work on <code>feature/sprint4-*</code> branches
API Docs	Swagger/OpenAPI	springdoc-openapi extended with FR-6 insights and admin endpoints; interactive UI at <code>/swagger-ui.html</code>
IDE	IntelliJ + VSCode	IntelliJ for Spring Boot; VS Code with Angular Language Service for frontend

4.2 Aggregation Query Design

Sprint 4 introduces no new DDL scripts. All analytics data is derived from the existing 10-table schema using optimised read queries. Key aggregation patterns are defined below to guide backend implementation.

4.2.1 Financial Insights Queries

- totalIncome: `SELECT SUM(t.amount) FROM transactions t JOIN accounts a ON t.account_id = a.id WHERE a.user_id = :userId AND t.type = 'CREDIT' AND a.is_active = true`
- totalExpense: `SELECT SUM(t.amount) FROM transactions t JOIN accounts a ON t.account_id = a.id WHERE a.user_id = :userId AND t.type = 'DEBIT' AND a.is_active = true`
- trendSummary: `GROUP BY YEAR(t.transaction_date), MONTH(t.transaction_date)` with separate SUM for CREDIT and DEBIT; ordered by year and month descending, limited to last 6 months
- All insight queries annotated with `@Query` and executed within `@Transactional(readOnly = true)` service methods for optimal performance

4.2.2 Admin Dashboard Queries

- totalUsers: `SELECT COUNT(*) FROM users`
- totalActiveUsers: `SELECT COUNT(*) FROM users WHERE is_active = true`
- totalLoans: `SELECT COUNT(*) FROM loan_applications`
- pendingApprovals: `SELECT COUNT(*) FROM loan_applications WHERE status = 'PENDING'`

- ▶ totalTransactions: SELECT COUNT(*) FROM transactions
- ▶ All admin aggregation queries executed within @Transactional(readOnly = true) for performance; no result caching in this sprint

5. INSTALLATION & SETUP — DAY 31

Day 31 is dedicated to Sprint 4 environment extension and analytics preparation. Both sessions must be completed before Day 32 development begins. No DDL scripts are required — the existing 10-table schema from Sprints 1–3 is sufficient.

5.1 Session 1: Sprint 4 Branch & Charting Library Setup

Git Branch Setup

- ▶ Create a new Git branch from develop: `git checkout -b feature/sprint4-insights-admin`
- ▶ Push the branch to remote: `git push -u origin feature/sprint4-insights-admin`
- ▶ Share branch access with all team members and the PMIS mentor

Angular Charting Library Installation

- ▶ Install ng2-charts and Chart.js: `npm install ng2-charts chart.js`
- ▶ Import NgChartsModule in AppModule: `import { NgChartsModule } from 'ng2-charts'`
- ▶ Verify installation: create a throwaway test component with a BaseChartDirective to confirm rendering
- ▶ Install Angular Material if not already present: `ng add @angular/material`
- ▶ Import MatCardModule for KPI summary cards in AppModule
- ▶ Run `ng serve` to confirm the Angular application starts without errors after all additions

5.2 Session 2: Backend Aggregation Layer Preparation

Spring Boot Repository & Service Scaffolding

- ▶ Create InsightsRepository interface with `@Query` methods for `totalIncome`, `totalExpense`, and `trendSummary` aggregations
- ▶ Create AdminDashboardRepository interface with `@Query` methods for all platform-wide metrics
- ▶ Create InsightsService and AdminDashboardService classes — all methods annotated `@Transactional(readOnly = true)`
- ▶ Create InsightsController and AdminDashboardController — apply `@PreAuthorize` annotations at method level
- ▶ Create DTOs: `FinancialInsightsDTO`, `TrendEntryDTO`, `AdminDashboardDTO`, `PendingApprovalDTO`, `UserActivityDTO`
- ▶ Run the Spring Boot application and verify all new beans load without errors before Day 32 development begins

6. SPRINT 4 EXECUTION PLAN

The 10-day sprint enables parallel backend and frontend development. An integration session on Day 38 bridges both layers. Each session is approximately 3–4 hours of focused work.

6.1 Sprint Schedule

DAY	PHASE	SESSION 1	SESSION 2
31	Env Setup	Create Sprint 4 Git branch; install ng2-charts, Chart.js, and Angular Material card modules	Scaffold InsightsRepository, AdminDashboardRepository, service classes, controllers, and all DTOs
32	Insights Backend	GET /api/insights/{userId} — endpoint with JWT userId cross-check	Aggregation logic: totalIncome, totalExpense, savings, and 6-month trendSummary; inactive account filtering
33	Insights UI	Angular financial summary cards: Total Income, Total Expense, Savings with loading skeleton	Chart.js bar/line chart rendering trendSummary month-by-month income vs. expense overlay
34	Admin Backend	GET /api/admin/dashboard — @PreAuthorize ADMIN only; platform metric aggregations	pendingApprovals count aggregation across modules; GET /api/admin/pending-approvals detail endpoint
35	Admin UI Layout	Angular Admin layout component — distinct nav, header, and route guards for /admin paths	Admin KPI summary cards: totalUsers, totalLoans, pendingApprovals, totalTransactions
36	System Monitor	GET /api/admin/pending-approvals — detailed lists across all pending-state modules	Angular data grids for pending approvals; actionable rows linking to decision workflows from Sprint 3
37	Admin Controls	GET /api/admin/users; PATCH /api/admin/users/{id}/status; GET /api/admin/system-health	Angular admin user management table; activate/deactivate controls; system health status display
38	Integration	Wire Angular insights dashboard to GET /api/insights/{userId}; verify chart updates on data load	Connect Angular Admin components to all admin APIs; verify JWT ADMIN role claim authorises data fetch
39	QA	JUnit: insights accuracy, inactive account exclusion, savings formula, admin 403 security tests	E2E frontend testing: chart rendering across screen sizes; admin control functionality validation
40	Wrap-Up	Update Swagger/OpenAPI for all new /api/insights/... and /api/admin/... endpoints	Final dashboard performance review; query optimisation check; QnA session; sprint sign-off

6.2 Day 32: Financial Insights Engine

Session 1 — Backend: GET /api/insights/{userId}

- ▶ InsightsController.getInsights(@PathVariable Long userId): extract JWT userId from SecurityContextHolder → compare with path userId → HTTP 403 if mismatch
- ▶ InsightsService.buildInsights(userId): call repository aggregation methods → assemble FinancialInsightsDTO → return
- ▶ @GetMapping("/api/insights/{userId}") secured with @PreAuthorize("isAuthenticated()")

Session 2 — Backend: Aggregation Logic

- ▶ `InsightsRepository.getTotalIncome(userId)`: @Query with SUM on CREDIT transactions filtered by active accounts
- ▶ `InsightsRepository.getTotalExpense(userId)`: @Query with SUM on DEBIT transactions filtered by active accounts
- ▶ `InsightsRepository.getTrendSummary(userId)`: @Query with GROUP BY YEAR, MONTH; return list of TrendEntryDTO covering last 6 months; pad missing months with zero values in service layer
- ▶ savingsAmount computed in service: `totalIncome.subtract(totalExpense)` — may be negative, returned as-is

6.3 Day 33: User Analytics Dashboard

Session 1 — Frontend: Summary Cards

- ▶ ng generate component insights/dashboard
- ▶ OnInit: call `InsightsService.getInsights(userId)` → populate component state
- ▶ Three Angular Material mat-card components: Total Income (green accent), Total Expense (red accent), Savings (blue accent or red if negative)
- ▶ Loading skeleton (mat-progress-bar or shimmer divs) displayed while API call is in-flight

Session 2 — Frontend: Trend Chart

- ▶ Integrate BaseChartDirective from ng2-charts with type 'bar' or 'line'
- ▶ datasets: [{ label: 'Income', data: trendIncome }, { label: 'Expense', data: trendExpense }]
- ▶ labels: month labels extracted from trendSummary (e.g., 'Nov 2025', 'Dec 2025')
- ▶ Chart colours: Income → green (#2E7D32), Expense → red (#C62828)
- ▶ Chart updates via Angular change detection when data observable emits

6.4 Day 34: Admin Dashboard Engine

Session 1 — Backend: GET /api/admin/dashboard

- ▶ @GetMapping("/api/admin/dashboard") with @PreAuthorize("hasRole('ADMIN')")
- ▶ AdminDashboardService.buildDashboard(): execute all aggregation queries → assemble AdminDashboardDTO
- ▶ AdminDashboardDTO fields: totalUsers, totalActiveUsers, totalLoans, pendingApprovals, totalTransactions, platformSavingsRate
- ▶ platformSavingsRate computed as: $(totalIncome - totalExpense) / totalIncome \times 100$ across all active accounts platform-wide

Session 2 — Backend: Pending Approvals Detail

- ▶ GET /api/admin/pending-approvals — @PreAuthorize("hasRole('ADMIN')")
- ▶ Returns list of PendingApprovalDTO: id, type (LOAN_APPLICATION), applicantName, productName, requestedAmount, appliedAt
- ▶ Filtered from loan_applications WHERE status = 'PENDING'; ordered by applied_at ascending (oldest first)

6.5 Day 35: Admin Dashboard Layout

Session 1 — Frontend: Admin Module Structure

- ▶ ng generate module admin --routing — creates a lazy-loaded admin module
- ▶ AdminLayoutComponent: dedicated sidenav with links to Dashboard, Pending Approvals, User Management, System Health

- ▶ AdminAuthGuard: CanActivate guard that checks decoded JWT for ADMIN role → redirects to /dashboard if CUSTOMER
- ▶ All /admin/** routes protected by AdminAuthGuard in admin-routing.module.ts

Session 2 — Frontend: KPI Summary Cards

- ▶ ng generate component admin/dashboard
- ▶ 4 mat-card KPI cards: Total Users, Total Loans, Pending Approvals (with badge), Total Transactions
- ▶ Pending Approvals card styled with amber/orange accent when count > 0 to draw admin attention
- ▶ Auto-refresh mechanism: setInterval polling every 60 seconds to keep admin metrics current

6.6 Day 36: System Monitoring & Pending Approvals UI

Session 1 — Backend: Pending Approvals Refinement

- ▶ Extend GET /api/admin/pending-approvals to support query param ?module=LOAN for module-filtered views
- ▶ GET /api/admin/system-health returns: { dbStatus: 'UP'|'DOWN', activeSessions: N, serverUptimeSeconds: N }
- ▶ dbStatus determined by executing a simple SELECT 1 health-check query

Session 2 — Frontend: Pending Approvals Grid

- ▶ ng generate component admin/pending-approvals
- ▶ Angular Material data table: columns — Type, Applicant, Product/Item, Amount, Applied Date, Action
- ▶ Action column: 'Review' button navigates to the loan decision component from Sprint 3
- ▶ Empty state: friendly message when no pending approvals exist

6.7 Day 37: Advanced Admin Controls

Session 1 — Backend: User Management & Activity APIs

- ▶ GET /api/admin/users — paginated list of all users; response excludes password_hash
- ▶ PATCH /api/admin/users/{userId}/status — payload: { isActive: true|false }; validation: admin cannot deactivate own account (HTTP 400)
- ▶ GET /api/admin/users/{userId}/activity — returns last 20 transactions and last 5 login events for the specified user
- ▶ All write operations append an audit log entry to an audit_log table or application log with actingAdminId, action, targetUserId, and timestamp

Session 2 — Frontend: User Management UI

- ▶ ng generate component admin/user-management
- ▶ Angular Material paginated table: columns — User ID, Full Name, Email, Role, Status, Registered Date, Actions
- ▶ Actions column: toggle Activate/Deactivate button; confirmation dialog before status change
- ▶ Status badge: ACTIVE → green chip, INACTIVE → red chip
- ▶ System health panel displays dbStatus with green/red indicator, active sessions count, and server uptime

7. API REFERENCE

All secured endpoints require a valid JWT Bearer token in the Authorization header. Endpoints inherited from Sprints 1–3 remain unchanged and are not repeated here.

FR-6 — FINANCIAL INSIGHTS ENDPOINTS

ENDPOINT	METHOD	DAY	AUTH	DESCRIPTION
GET /api/insights/{userId}	GET	Day 32	JWT	Returns totalIncome, totalExpense, savings, and 6-month trendSummary for the authenticated user. 403 if userId mismatches JWT.

FR-6 — ADMIN DASHBOARD ENDPOINTS

ENDPOINT	METHOD	DAY	AUTH	DESCRIPTION
GET /api/admin/dashboard	GET	Day 34	ADMIN	Platform-wide metrics: totalUsers, totalLoans, pendingApprovals, totalTransactions, platformSavingsRate.
GET /api/admin/pending-approvals	GET	Day 34	ADMIN	Detailed list of all PENDING approval items across modules. Supports ?module= filter. Ordered oldest-first.
GET /api/admin/system-health	GET	Day 36	ADMIN	Returns DB connectivity status, active session count, and server uptime in seconds.

FR-6 — ADMIN USER MANAGEMENT ENDPOINTS

ENDPOINT	METHOD	DAY	AUTH	DESCRIPTION
GET /api/admin/users	GET	Day 37	ADMIN	Paginated list of all registered users. Password hashes never included in response.
PATCH /api/admin/users/{userId}/status	PATCH	Day 37	ADMIN	Activate or deactivate a user account. Admin cannot deactivate own account (HTTP 400).
GET /api/admin/users/{userId}/activity	GET	Day 37	ADMIN	Returns last 20 transactions and last 5 login events for the specified user.

8. INTEGRATION STRATEGY

The integration session on Day 38 bridges Angular and Spring Boot for all Sprint 4 modules. This session validates end-to-end data flow from the user interface through the API layer to MySQL and back, building on the JWT infrastructure verified in all prior sprints.

8.1 Day 38 — Sprint 4 Full Integration

Session 1 — Insights Dashboard Integration

- ▶ Wire Angular InsightsService to GET /api/insights/{userId}; verify HTTP Interceptor attaches Bearer token
- ▶ Test cross-user access from Angular: manually replace userId in URL with another user's ID → verify HTTP 403 displayed and no data leaked
- ▶ Verify chart renders immediately on page load with data from API response — no manual refresh required
- ▶ Test inactive account exclusion: deactivate a test account via admin API → re-fetch insights → verify deactivated account's transactions absent from totals
- ▶ Verify negative savings renders correctly in the Savings card (e.g., red text or downward indicator)
- ▶ Confirm loading skeleton appears while API call is in-flight and disappears on response

Session 2 — Admin Dashboard Integration

- ▶ Connect Angular AdminDashboardService to GET /api/admin/dashboard; verify ADMIN JWT claim authorises the fetch
- ▶ Test CUSTOMER token against GET /api/admin/dashboard → assert HTTP 403 displayed in Angular and user redirected
- ▶ Verify KPI card values in Angular match direct SQL COUNT queries on the database
- ▶ Test pending approvals data grid: create 2 PENDING loan applications → verify they appear in GET /api/admin/pending-approvals → confirm grid renders both rows
- ▶ Test user status toggle: deactivate customer1 via Angular admin controls → verify is_active = false in DB → verify customer1 cannot login
- ▶ Verify system health panel: displays 'UP' status when DB is reachable; correct session count and uptime displayed

9. DATA SEEDING

SQL seeding scripts generate historical transaction data across multiple months for Sprint 4 testing. All Sprint 1–3 seed data is preserved and extended. No new tables are seeded — existing tables are populated with additional records.

ENTITY	COUNT	SEED DETAILS
Transaction History	60+	10+ CREDIT and DEBIT transactions per month for the last 6 months for customer1 and customer2 — ensures trendSummary has meaningful chart data across all months
Inactive Account	1	One account for customer3 set is_active = false — used to verify inactive account exclusion from insight calculations
Pending Loans	3+	3 additional PENDING loan applications seeded (beyond Sprint 3 data) to test the pendingApprovals count and detail grid on the Admin Dashboard
User Accounts	5+	Total of at least 5 user records (1 ADMIN + 4 CUSTOMER); mix of active and inactive to test totalUsers vs. totalActiveUsers metrics
Test Credentials	N/A	admin@neobank.in / Admin@123 (ADMIN role) and customer1@neobank.in / Customer@123 (CUSTOMER role) from Sprint 1 seed retained and active

10. QUALITY ASSURANCE & TESTING

QA is integrated into Day 39. Testing covers the complete Insights and Admin Controls module with a particular emphasis on aggregation accuracy, inactive account exclusion, and strict enforcement of Admin-only access controls. All tests must pass before sprint sign-off on Day 40.

DAY 39 — FR-6 INSIGHTS QA

Unit Tests

- ▶ `totalIncome` accuracy: seed 5 CREDIT transactions of known amounts for a test user → assert API returns exact sum
- ▶ `totalExpense` accuracy: seed 5 DEBIT transactions of known amounts → assert API returns exact sum
- ▶ Savings formula: assert `savings = totalIncome - totalExpense` for both positive and negative cases
- ▶ Inactive account exclusion: seed CREDIT and DEBIT transactions on an inactive account → assert they do NOT appear in `totalIncome` or `totalExpense`
- ▶ `trendSummary` month count: assert exactly 6 entries returned even when some months have zero transactions
- ▶ `trendSummary` zero-padding: month with no transactions must appear with `income = 0.00` and `expense = 0.00`, not be absent
- ▶ Cross-user access: submit GET `/api/insights/{otherUserId}` with a different user's JWT → assert HTTP 403 Forbidden
- ▶ Unauthenticated access: omit Bearer token → assert HTTP 401 Unauthorized

End-to-End Tests

- ▶ Login as `customer1` → navigate to `/insights` → verify 3 summary cards render with non-zero values matching DB aggregation
- ▶ Verify `Chart.js` chart renders with 6 bars/points on the `trendSummary` chart
- ▶ Verify negative savings scenario: ensure expense-heavy test user sees Savings card rendered with red styling
- ▶ Verify loading skeleton appears and disappears correctly during API call lifecycle

DAY 39 — FR-6 ADMIN CONTROLS QA

Unit Tests

- ▶ Admin dashboard — CUSTOMER token: assert HTTP 403 Forbidden on GET `/api/admin/dashboard`
- ▶ Admin dashboard — unauthenticated: assert HTTP 401 Unauthorized
- ▶ `totalUsers` count: assert `COUNT(*)` in API response matches `SELECT COUNT(*) FROM users` in DB
- ▶ `pendingApprovals` count: assert value matches `SELECT COUNT(*) FROM loan_applications WHERE status = 'PENDING'` in DB
- ▶ User deactivation — self-deactivation: admin attempts to deactivate own account → assert HTTP 400 Bad Request
- ▶ User deactivation: deactivate a CUSTOMER user → assert `is_active = false` in DB → assert that user receives HTTP 403 on next login attempt
- ▶ Admin user listing — no credentials: assert `password_hash` field absent from every user object in response
- ▶ System health endpoint: assert `dbStatus = 'UP'` when DB is reachable; assert HTTP 403 for non-admin access

End-to-End Tests

- ▶ Login as admin@neobank.in → navigate to /admin/dashboard → verify all KPI cards display correct values matching DB
- ▶ Pending approvals grid: verify 3 seeded PENDING loans appear in the grid with correct applicant and amount
- ▶ User management: deactivate customer3 from Angular admin UI → attempt login as customer3 → verify HTTP 403 returned
- ▶ CUSTOMER route guard: login as customer1 → manually navigate to /admin/dashboard → verify Angular route guard redirects to /dashboard

11. DOCUMENTATION & SPRINT WRAP-UP

Day 40 concludes with a documentation, performance review, and sign-off session. All deliverables below must be committed to Git before the sprint is considered closed.

11.1 API Documentation

- ▶ Extend springdoc-openapi annotations to cover all new FR-6 endpoints under `/api/insights/...` and `/api/admin/...`
- ▶ Annotate `InsightsController` and `AdminDashboardController` with `@Operation` and `@ApiResponse` for each method
- ▶ Annotate `FinancialInsightsDTO`, `TrendEntryDTO`, `AdminDashboardDTO`, `PendingApprovalDTO` with `@Schema` including field descriptions and example values
- ▶ Verify Swagger UI at `http://localhost:8080/swagger-ui.html` reflects all 7 new endpoints with correct request/response schemas and role requirements
- ▶ Export OpenAPI JSON: `http://localhost:8080/v3/api-docs` → commit as `docs/openapi-sprint4.json`

11.2 Frontend Documentation

- ▶ Document all new Angular components: `insights/dashboard`, `admin/dashboard`, `admin/pending-approvals`, `admin/user-management`
- ▶ Document `InsightsService` and `AdminDashboardService`: methods, HTTP calls, chart data transformation logic
- ▶ Update routing module documentation to include `/insights`, `/admin/dashboard`, `/admin/pending-approvals`, `/admin/users` routes and their respective guards
- ▶ Update `README.md` in `neobank-frontend` with Sprint 4 features, `ng2-charts` dependency, and Angular admin module structure

11.3 Performance Review

- ▶ Measure response time of `GET /api/insights/{userId}` with 6-month data — target: under 500ms
- ▶ Measure response time of `GET /api/admin/dashboard` — target: under 300ms
- ▶ Review SQL EXPLAIN output for aggregation queries — confirm indexes on `transaction_date`, `account_id`, and `type` columns are being utilised
- ▶ Review Angular Lighthouse score for the insights dashboard — target: Performance > 80

11.4 Sprint Review & Sign-Off

- ▶ Live demonstration of all Sprint 4 deliverables against acceptance criteria in this SRS
- ▶ Code review of both repositories with PMIS mentor — focus on aggregation query correctness, inactive account filtering, and `@PreAuthorize` security
- ▶ QnA session for Lab 4 and Lab 5 intern teams
- ▶ Formal mentor sign-off upon acceptance of all deliverables
- ▶ Workspace preparation for Milestone 7: Stabilization & Deployment — the final sprint of the NeoBank 360 project

12. ACCEPTANCE CRITERIA

All criteria below constitute the Definition of Done for Sprint 4. Every item must be met before the sprint can be signed off.

FR-6 — FINANCIAL INSIGHTS

#	CRITERION	VERIFIED BY
AC01	GET /api/insights/{userId} returns correct totalIncome, totalExpense, and savings values matching DB aggregation	Unit Test
AC02	Inactive account transactions strictly excluded from all insight calculations	Unit Test
AC03	trendSummary returns exactly 6 monthly entries; months with zero transactions included with zero values	Unit Test + Postman
AC04	Savings value correctly computed as totalIncome – totalExpense; negative savings returned as-is	Unit Test
AC05	Cross-user access to insights rejected with HTTP 403 Forbidden	Unit Test
AC06	Angular insights dashboard renders summary cards and Chart.js trend chart with live API data	E2E Test
AC07	Loading skeleton displayed during API call; chart updates without full page reload	E2E Test

FR-6 — ADMIN DASHBOARD & CONTROLS

#	CRITERION	VERIFIED BY
AC08	GET /api/admin/dashboard restricted to ADMIN role; CUSTOMER or unauthenticated access returns 403/401	Unit Test
AC09	Admin dashboard metrics (totalUsers, totalLoans, pendingApprovals) match live DB counts	Unit Test + Postman
AC10	GET /api/admin/pending-approvals returns correct list of PENDING items ordered oldest-first	Unit Test + Postman
AC11	PATCH /api/admin/users/{userId}/status correctly toggles user is_active; self-deactivation returns HTTP 400	Unit Test
AC12	Admin user listing never exposes password_hash or sensitive credential fields	Unit Test + Postman
AC13	GET /api/admin/system-health returns DB status, session count, and uptime; CUSTOMER access returns 403	Unit Test
AC14	Angular Admin Dashboard KPI cards display values matching DB; AdminAuthGuard redirects CUSTOMER users	E2E Test
AC15	Angular pending approvals data grid renders seeded PENDING loans with correct applicant and amount data	E2E Test
AC16	Angular user management: deactivate/activate toggle works; deactivated user cannot subsequently login	E2E Test

GENERAL QUALITY

#	CRITERION	VERIFIED BY
AC17	All unit tests pass with zero critical failures across FR-6	Test Suite
AC18	Swagger UI accessible; all 7 new endpoints documented with request/response schemas and role annotations	Manual Review
AC19	GET /api/insights/{userId} responds in under 500ms with 6-month data; admin dashboard under 300ms	Performance Test
AC20	Git repositories clean: Sprint 4 feature branches merged, commit messages descriptive	Mentor Review
AC21	No hardcoded credentials; aggregation queries reviewed for index utilisation; Infosys formatting standards met	Code Review
AC22	README.md files updated in both repos reflecting Sprint 4 features and ng2-charts dependency	Mentor Review

*Prepared under the PMIS (Pradhan Mantri Internship Scheme) · Infosys Bhubaneswar DC · Lab 4 & Lab 5
© Infosys Limited · Confidential · Not for External Distribution*